LAPORAN PRAKTIKUM PEKAN 2

STRUKTUR DATA

“ARRAY LIST”

disusun Oleh:

NABIL FIKRI

2511533011

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum mata kuliah **Struktur Data** ini tepat pada waktunya.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum mengenai implementasi kelas dan objek melalui studi kasus pembuatan ArrayList menggunakan bahasa pemrograman Java. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada:

1. Dr. Wahyudi S.T,M.T, selaku dosen mata kuliah yang telah memberikan bimbingan teori di kelas.
2. Kakak asisten praktikum yang telah memberikan arahan dan bantuan teknis selama proses praktikum berlangsung di laboratorium.
3. Rekan-rekan mahasiswa angkatan 2026 Informatika Universitas Andalas yang telah saling mendukung dalam proses belajar.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi materi maupun sistematika penulisannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan perkembangan ilmu pengetahuan, khususnya di bidang Informatika.

Padang, 17 April 2026

Nabil Fikri

DAFTAR ISI

KATA PENGANTAR i

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

BAB II PEMBAHASAN 2

2.1 Mengenal ArrayList 2

2.2 Karakteristik Utama ArrayList 2

2.3 Operasi Dasar pada ArrayList 3

2.4 Analisis Class 4

2.1.1 ImplemeClass ArrayList1_2511533011 4

2.1.2 Analisis Class DaftarKata_2511533011 dan Driver 5

2.1.3 Implementasi pada class Mahasiswa_ 2511533011 dan Driver 7

BAB III PENUTUP 10

3.1 Kesimpulan 10

3.2 Saran 11

DAFTAR PUSTAKA 12

BAB I
PENDAHULUAN

1.1 Latar Belakang

ArrayList merupakan array dinamis yang memungkinkan penambahan elemen tanpa batas, berbeda dengan array yang memiliki ukuran tetap. ArrayList di Java adalah untuk mengatasi keterbatasan **array konvensional (statis)** yang ukurannya tidak dapat diubah setelah dideklarasikan. ArrayList hadir sebagai solusi dinamis dalam Java Collection Framework yang memudahkan penambahan, penghapusan, dan manajemen data tanpa perlu menentukan ukuran awal.

1.2 Tujuan Praktikum

1. Memberi pemahaman bagaimana mendeklarasikan suatu array list\
2. Memasukkan data ke dalam suatu array list
3. Mengakses array list
4. Mengimplementasikannya untuk menyelesaikan suatu permasalahannya

BAB II
PEMBAHASAN

2.1 Mengenal ArrayList

ArrayList adalah struktur data yang digunakan untuk menyimpan kumpulan data dalam bahasa Java. Berbeda dengan tipe data array biasa, ArrayList memiliki ukuran yang dinamis, di mana jumlah element array dapat berubah sesuai kebutuhan. ArrayList juga dapat menyimpan data dengan tipe data yang berbeda, sedangkan array biasanya hanya bisa menyimpan data dalam 1 jenis tipe data saja. Dengan keunggulan ini, ArrayList lebih efisien untuk operasi penambahan dan penghapusan elemen, terutama di tengah-tengah list.

Terdapat beberapa kelebihan dan kekurangan pada ArrayList :

Kelebihan :

1. Ukurannya yang dinamis.
2. Dapat menyimpan data dengan tipe data yang berbeda.
3. Operasi penambahan dan penghapusan elemen relatif cepat.

Kekurangan :

1. Pengaksesan elemen secara acak relatif lambat.

Dapat diartikan bahwa ArrayList memiliki kinerja sedikit lebih lambat dalam scenario tertentu (karena manajemen memori dinamis), namun jauh lebih efisien. Sedangkan, Array biasa yang memiliki kinerja cepat, tapi kaku (ukuran tetap).

2.2 Karakteristik Utama ArrayList

Berdasarkan praktikum yang dilakukan, ArrayList memiliki karakteristik sebagai berikut:

- **Pengindeksan:** Setiap elemen disimpan berdasarkan indeks yang dimulai dari 0.
- **Penyimpanan Objek:** ArrayList hanya dapat menyimpan tipe data objek (referensi). Untuk menyimpan tipe data primitif seperti int atau double, Java

menggunakan mekanisme *autoboxing* ke *wrapper class* (seperti Integer atau Double).

- **Urutan Elemen (Ordered):** Elemen-elemen dalam ArrayList tetap tersusun sesuai dengan urutan saat elemen tersebut dimasukkan.\

2.3 Operasi Dasar pada ArrayList

Dalam praktikum pekan ke-2, terdapat beberapa operasi fundamental yang diimplementasikan:

1. **Penambahan (add):** Digunakan untuk memasukkan elemen ke dalam list. Operasi ini dapat dilakukan di akhir list atau pada indeks tertentu (penyisipan/insertion).
2. **Pengubahan (set):** Digunakan untuk mengganti nilai pada posisi indeks tertentu tanpa mengubah ukuran list.
3. **Penghapusan (remove):** Digunakan untuk menghapus elemen berdasarkan indeks. Saat elemen dihapus, elemen-elemen di belakangnya akan bergeser ke kiri (*shifting*) untuk mengisi kekosongan.
4. **Akses (get):** Digunakan untuk mengambil nilai elemen pada posisi tertentu dengan kompleksitas waktu $O(1)$.

2.4 Analisis Class

2.1.1 ImplemeClass ArrayList1_2511533011

```java
package pekan2_2511533011;
import java.util.ArrayList;

public class ArrayList1_2511533011 {
public static void main(String[] args) {
// Size of the ArrayList
    int n = 5;
    // Declaring the ArrayList with initial size n
    ArrayList<Integer> arrli = new ArrayList<Integer>(n);
    // Appending new elements at the end of List
    for (int i = 1; i <= n; i++)
    arrli.add(i);
    // Printing elements
    System.out.println(arrli);
    // Remove element at index 3
    arrli.remove(3);
    //Displaying the ArrayList
    // after deletion
    System.out.println(arrli);
    // Printing elements one by one
    for (int i = 0; i < arrli.size(); i++)
    System.out.print(arrli.get(i) + " ");

}
}
```

Gambar 2.1.1

Pada class ini mendeklarasikan **ArrayList** yang menampung tipe data **Int** dengan kapasitas awal adalah 5. Meskipun kapasitas awalnya 5, ukurannya tetap fleksibel. Pada bagian perulangan **for,** method **.add(i)** menunjukan bahwa **ArrayList** mengisi elemen secara berurutan. Karena **i** dimulai dari 1 sampai 5, maka isi awal dari list adalah [1, 2, 3, 4, 5].

Sementara itu, method **arrli.remove(3)** menjelaskan bahwa ini berfungsi untuk menghapus elemen pada indeks ke-3 yang di mana hasilnya [1, 2, 3, 5], karena angka 4 merupakan nilai dari indeks ke-3. Maka dari itu, angka 5 yang tadinya berada pada indeks ke-4 bergeser ke indeks 3. Pergeseran ini disebut dengan *Shifting* elemen.

Selanjutnya, kombinasi antara **.size()** untuk mengetahui jumlah elemen saat ini dan **.get(i)** untuk mengambil nilai berdasarkan indeks. Ini menunjukkan cara

manual mengakses elemen satu per satu, selain menggunakan **System.out.println(arrli)** secara langsung.

2.1.2 Analisis Class DaftarKata_2511533011 dan Driver

```java
public class DaftarKata_2511533011 {
    private final ArrayList<String> data;
    // Konstruktor : inisialisasi list kosong
    public DaftarKata_2511533011() {
        this.data = new ArrayList<>();
    }
    /** Menentukan elemen di akhir list. */
    public void tambah_2511533011(String elemen) {
        data.add(elemen);
    }
    /** Menambahkan elemen pada indeks tertentu (menyisipkan). */
    public void tambahPada_2511533011(int index, String elemen) {
        data.add(index, elemen);
    }
    /**
     * Mengubah elemen pada posisi 'index' menjadi 'nilaBaru'.
     * Bertindak sebagai "setter" untuk elemen tertentu.
     */
    public void ubahElemen_2511533011(int index, String nilaiBaru) {
        data.set(index, nilaiBaru);
    }
    /**
     * Menghapus elemen pada posisi 'index' dan mengembalikan nilai yang dihapus.
     */
    public String hapusElemen_2511533011(int index) {
        return data.remove(index);
    }

    public void iterasiCetak_2511533011() {
        for (int i = 0; i < data.size(); i++) {
            System.out.print(data.get(i)+" ");
        }
    }

    /** mengambil elemen berdasarkan indeks. */
    public String get(int index) {
        return data.get(index);
    }

    // Representasi String agar mudah dicetak
    @Override
    public String toString() {
        return data.toString();
```

Gambar 2.1.2

Pada bagian ini, terdapat penerapan enkapsulasi yaitu menggunakan akses **private** (Enkapsulasi) pada variable data yang menunjukkan bahwa data di dalam list tidak bisa diakses langsung dari luar class, melainkan harus melalui metode (seperti **tambah_2511533011**). Ini menjaga keamanan data agar tidak dimodifikasi secara sembarangan.

Terdapat dua metod berbeda untuk menambahkan kata :

- **tambah_2511533011** yang menggunakan **data.add(elemen).** Secara ototmatis data masuk ke urutan paling akhir.
- **tambahPada_2511533011** yang menggunakan **data.add(index, elemen)**. Ini adalah operasi penyisipan.

Selain itu, terdapat metode **ubahElemen_2511533011** (Transformasi data) menggunakan **.set()**. Berbeda dengan **add, set()** tidak menambah jumlah elemen. Ia hanya menimpa (*overwrite*) nilai lama pada indeks tertentu dengan nilai baru.

Terakhir, pada syntax (Penghapusan dan Return Value)

```
public String hapusElemen_2511533011(int index) {
   return data.remove(index);
}
```

metode ini mengembalikan nilai **String**. Ini memungkinkan class **Driver** untuk mengetahui kata apa yang baru saja dihapus dan menampilkannya ke layar (seperti pada variabel **terhapus**)

Selanjutnya analisis class **DaftarKataDriver_2511533011**

```java
package pekan2_2511533011;

public class DaftarKataDriver_2511533011 {
    public static void main(String[] args) {
        DaftarKata_2511533011 al = new DaftarKata_2511533011();

        // Menambah elemen (akhir)
        al.tambah_2511533011("Kami");
        al.tambah_2511533011("Informatika");

        //Menyisipkan elemen pada indeks 1
        al.tambahPada_2511533011(1, "Mahasiswa");

        // Cetak isi awal
        System.out.println("Awal            : " + al);

        // Mengubah elemen (index 1)
        al.ubahElemen_2511533011(1, "Departemen");
        System.out.println("Setelah Ubah        :" + al);

        // Menghapus elemen (index 1)
        String terhapus = al.hapusElemen_2511533011(0);
        System.out.println("Terhapus        : " + terhapus);
        System.out.println("Setelah Hapus       : " + al);

        // Iterasi pada ArrayList (cetak setiap elemen)
        System.out.print("Iterasi            :");
        al.iterasiCetak_2511533011();
        System.out.println();
    }

}
```

Pada class ini hanya menjelaskan alur eksekusi program yang mana objek **al** dibuat sebagai Inisialisasi, lalu menunjukkan transisi dari kalimat dari *"Kami*

Informatika" menjadi *"Kami Mahasiswa Informatika",* lalu mengubahnya lagi menjadi *"Kami Departemen Informatika".* Hasil akhir setelah penghapusan indeks 0 ("Kami"), sisa list adalah *"Departemen Informatika".*

2.1.3 Implementasi pada class Mahasiswa_ 2511533011 dan Driver

```java
package pekan2_2511533011;

public class Mahasiswa_2511533011 {
    String nim;
    String nama;
    String prodi;
    Mahasiswa_2511533011(String nim, String nama, String prodi) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;;
    }
    @Override
    public String toString() {
        return "NIM : " + nim + ", Nama : " + nama + ", Prodi : " + prodi;
    }

}
```

Class ini bagian paling kompleks yang mana penggunaan **ArrayList** tidak hanya bisa menyimpan tipe data dasar (seperti **Integer** atau **String**), tetapi juga bisa menyimpan referensi ke objek dari class buatan sendiri. Ini memungkinkan penyimpanan data yang lebih terstruktur (NIM, Nama, dan Prodi terikat dalam satu kesatuan).

Terdapat metode **@Override public String toString()** (Overriding) yang mana tanpa metode ini, saat mencetak objek mahasiswa, Java hanya menampilkan alamat memori (hashcode). Dengan melakukan *override,* data mahasiswa lebih mudah dibaca saat dipanggil oleh **System.out.println(mhs).**

Selanjutnya analisis class **MahasiswaDriver_2511533011**

```java
package pekan2_2511533011;
import java.util.ArrayList;

public class MahasiswaDriver_2511533011 {
    // 1. Method untuk menampilkan menu
    public static void tampilkanMenu_2511533011() {
        System.out.println(" \nMenu : ");
        System.out.println("1. Tambahkan Mahasiswa");
        System.out.println("2. Tampilkan Semua Mahasiswa");
        System.out.println("3. Hapus Mahasiswa Berdasarkan NIM");
        System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
        System.out.println("5. Keluar");
    }

    // 2. Method untuk tambah Mahasiswa
    public static void tambahMahasiswa_2511533011(ArrayList<Mahasiswa_2511533011> list, Scanner sc) {
        System.out.println("Masukkan NIM : ");
                String nim = sc.nextLine();
                System.out.println("Masukkan Nama : ");
                String nama = sc.nextLine();
                System.out.println("Masukkan Prodi : ");
                String prodi = sc.nextLine();
                list.add(new Mahasiswa_2511533011(nim, nama, prodi));
                System.out.println("mahasiswa berhasil ditambahkan");
    }

    // 3. Method untuk menampilkan semua data
    public static void tampilkanSemuaMahasiswa_2511533011(ArrayList<Mahasiswa_2511533011> list) {
        if (list.isEmpty()) {
            System.out.println("daftar mahasiswa kosong.");
        } else {
            System.out.println("Data Mahasiswa : ");
            for (Mahasiswa_2511533011 mhs : list) {
                System.out.println(mhs);
            }
        }
    }

    // 4. Method untuk hapus mahasiswa berdasarkan NIM
    public static void hapusMahasiswa_2511533011(ArrayList<Mahasiswa_2511533011> list, Scanner sc) {
        System.out.println("Masukkan NIM yang akan dihapus : ");
        String nimHapus = sc.nextLine();
        boolean removed = list.removeIf(mhs -> mhs.nim.equals(nimHapus));

        if (removed) {
            System.out.println("Data dengan NIM" + nimHapus + " berhasil dihapus. ");
        } else {
            System.out.println("NIM tidak ditemukan.");
        }
    }

    // 5. Method untuk mencari mahasiswa berdasarkan NIM
    public static void cariMahasiswa_2511533011(ArrayList<Mahasiswa_2511533011> list, Scanner sc) {
        System.out.print("Masukkan NIM yang dicari : ");
        String nimCari = sc.nextLine();
        boolean ditemukan = false;

        for (Mahasiswa_2511533011 mhs : list) {
            if (mhs.nim.equals(nimCari)) {
                System.out.println("Hasil Pencarian : " + mhs);
                ditemukan = true;
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("NIM tidak ada.");
        }
    }
```

```java
    public static void main(String[] args) {
        ArrayList<Mahasiswa_2511533011> mahasiswaList = new ArrayList<>();
        Scanner scanner = new Scanner(System.in);
        int choice;
        do {
            tampilkanMenu_2511533011();
            System.out.println("Pilih menu : ");
            choice = scanner.nextInt();
            scanner.nextLine(); // Consume newline

            switch (choice) {
            case 1:
                tambahMahasiswa_2511533011(mahasiswaList, scanner);
                break;
            case 2:
                tampilkanSemuaMahasiswa_2511533011(mahasiswaList);
                break;
            case 3:
                hapusMahasiswa_2511533011(mahasiswaList, scanner);
                break;
            case 4:
                cariMahasiswa_2511533011(mahasiswaList, scanner);
                break;
            case 5:
                System.out.println("Keluar dari program.");
                break;
            default:
                System.out.println("Pilihan tidak valid.");

            }

        }while (choice != 5);
        scanner.close();

    }
}
```

Gambar 2.1.3

Pada class driver ini, class menggunakan **Lambda Expression.** Syntax **list.removeIf(mhs -> mhs.nim.equals(nimHapus));** dengan metode **removeIf** (penggunaan Predicate) akan menelusuri seluruh list secara otomatis. Jika ada objek mahasiswa yang NIM nya cocok dengan **nimHapus,** maka objek tersebut akan dihapus. Ini jauh lebih efisien daripada menulis perulangan manual.

Pada metode **cariMahasiswa_2511533011** (Linear Search) menggunakan perulangan **for-each** di mana program akan membandingkan **nimCari** dengan setiap elemen di dalam **mahasiswaList**. Karena data tidak terurut berdasarkan NIM, metode yang digunakan adalah **Linear Search** dengan kompleksitas waktu *O(n)*.

Selanjutnya menambahkan **scanner.nextLine();** (Bug Fix) setelah **nextInt().** **nextInt()** hanya membaca angka dan meninggalkan karakter “Enter” (\n) di *buffer.* Tanpa baris tambahan ini, input teks berikutnya (NIM) akan terlewati secara otomatis oleh program

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum pekan ke-2 mengenai struktur data ArrayList, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Fleksibilitas Struktur Data:** ArrayList terbukti lebih efisien dibandingkan array statis dalam menangani data yang jumlahnya berubah-ubah (dinamis). Hal ini terlihat pada class ArrayList1 dan DaftarKata, di mana penambahan atau penghapusan elemen tidak memerlukan pendefinisian ukuran array ulang secara manual.

2. **Pentingnya Enkapsulasi:** Implementasi pada class DaftarKata menunjukkan bahwa membungkus ArrayList di dalam sebuah objek kustom meningkatkan keamanan data dan modularitas kode. Dengan menyediakan metode khusus seperti tambah_2511533011(), akses langsung ke data mentah dapat dibatasi.

3. **Pengolahan Objek Kustom:** ArrayList sangat kuat untuk mengelola basis data sederhana. Pada class Mahasiswa, terlihat bahwa ArrayList mampu menyimpan kumpulan objek kompleks dan memudahkan operasi CRUD (*Create, Read, Update, Delete*) melalui metode bawaan seperti removeIf() dan penggunaan perulangan *for-each*.

4. **Efisiensi Pencarian:** Operasi pencarian pada ArrayList yang tidak terurut (seperti pencarian mahasiswa berdasarkan NIM) menggunakan mekanisme *Linear Search* dengan kompleksitas waktu $O(n)$, yang berarti waktu pencarian berbanding lurus dengan jumlah data yang tersimpan.

3.2 Saran

Untuk pengembangan selanjutnya, disarankan agar implementasi ArrayList dilengkapi dengan mekanisme validasi data guna mencegah adanya input ganda (seperti NIM yang sama) serta penambahan fitur pengurutan menggunakan Collections.sort() untuk meningkatkan efisiensi pencarian. Selain itu, penerapan *exception handling* (seperti try-catch) sangat penting untuk mengantisipasi kesalahan input pengguna pada terminal, sementara pembandingan langsung dengan LinkedList perlu dilakukan untuk memahami perbedaan performa secara praktis dalam operasi penyisipan dan penghapusan data berskala besar.

DAFTAR PUSTAKA

[1] Duniailkom, "Pengertian ArrayList Bahasa Java beserta Contohnya," [Online]. Available: https://www.duniailkom.com/pengertian-arraylist-bahasa-java/. [Accessed 18 April 2026].

[2] AndreFikri, "Pengantar Array List dalam Java," SCRIBD, [Online]. Available: https://www.scribd.com/document/444295260/MODUL-1-Array-List#google_vignette. [Accessed 18 April 2026].

LAPORAN TUGAS

1. Deskripsi Tugas Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep ArrayList untuk mengelola sekumpulan data (objek) secara dinamis. Program terdiri dari dua bagian utama yaitu ADT (Abstract Data Type) sebagai cetak biru data musik, dan Driver sebagai pusat kendali program.

2. Aturan Penamaan (Wajib Dipatuhi)

1) Nama Kelas : Gunakan akhiran NIM lengkap. Contoh : Musik_2311532008 dan Playlist_2311532008.

2) Nama Variabel Atribut : Setiap variabel atribut di kelas ADT gunakan akhiran 4 digit terakhir NIM. Contoh (NIM ...2008) : String judul_2008, String penyanyi_2008, int durasi_2008.

3. Struktur Program

1) Kelas ADT Musik (Musik_NIMLengkap) Atribut: - Judul Lagu - - Penyanyi Durasi (dalam detik) Method : - Constructor untuk menginisialisasi atribut. - - Selektor (Getter) untuk setiap atribut. Mutator (Setter) untuk setiap atribut.

2) Kelas Driver (Playlist_NIMLengkap) - Tambah Lagu : Input data lagu baru ke ArrayList. - - - Lihat Playlist : Tampilkan seluruh daftar lagu (gunakan perulangan/looping). Hapus Lagu : Hapus lagu berdasarkan nomor indeks. Cek Kapasitas : Tampilkan jumlah total lagu menggunakan method .size().

4. Contoh Tampilan Output Program === Playlist Musik NIM: 2311532008 === 1. Tambah Lagu 2. Lihat Playlist 3. Hapus Lagu 4. Keluar Pilihan: 1 Masukkan Judul: Risk It All Masukkan Penyanyi: Bruno Mars Masukkan Durasi (detik): 220 Data berhasil ditambahkan!

1. Class Musik_2511533011

```java
package pekan2_2511533011;

public class Musik_2511533011 {
String JudulLagu_3011;
String Penyanyi_3011;
int Durasi_3011;

Musik_2511533011 (String JudulLagu, String Penyanyi, int Durasi) {
    this.JudulLagu_3011 = JudulLagu;
    this.Penyanyi_3011 = Penyanyi;
    this.Durasi_3011 = Durasi;
}
@Override
public String toString() {
    return "Judul : " + JudulLagu_3011 +"\n"+ "Nama Penyanyi : " + Penyanyi_3011 +"\n"+ "Durasi : " + Durasi_3011;
}

}
```

2. Class Playlist_251153301

```java
package pekan2_2511533011;
import java.util.ArrayList;
import java.util.Scanner;

public class Playlist_2511533011 {
    public static void tampilkanMenu_3011() {
        System.out.println("===Playlist Musik NIM : 2511533011===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Lihat Playlist");
        System.out.println("3. Hapus Lagu");
        System.out.println("4. keluar");
    }

    public static void tambahLagu_3011 (ArrayList<Musik_2511533011>list, Scanner input) {
        System.out.println("Masukkan Judul : ");
        String JudulLagu_3011 = input.nextLine();
        System.out.println("Masukkan Penyanyi : ");
        String Penyanyi_3011 = input.nextLine();
        System.out.println("Masukkan Durasi(detik) : ");
        int Durasi_3011 = input.nextInt();
        list.add(new Musik_2511533011(JudulLagu_3011, Penyanyi_3011, Durasi_3011));
        System.out.println("Data berhasil ditambahkan!");
    }

    public static void lihatPlaylist_3011 (ArrayList<Musik_2511533011>list) {
        if (list.isEmpty()) {
            System.out.println("Daftar Musik Kosong.");
        } else {
            System.out.println("Data Musik : ");
            for (Musik_2511533011 msk : list) {
                System.out.println(msk);
            }
        }
    }
    public static void hapusLagu_3011 (ArrayList<Musik_2511533011>list, Scanner input) {
        System.out.println("Masukkan judul lagu yang akan dihapus: ");
        String laguHapus = input.nextLine();
        boolean removed = list.removeIf(msk -> msk.JudulLagu_3011.equals(laguHapus));

        if (removed) {
            System.out.println("Data dengan Judul Lagu " + laguHapus + " berhasil dihapus.");
```

```java
        } else {
            System.out.println("Data tidak ditemukan.");
        }
    }
    public static void main(String[] args) {
        ArrayList<Musik_2511533011>musikList = new ArrayList<>();
        Scanner input = new Scanner(System.in);
        int choice;
        do {
            tampilkanMenu_3011();
            System.out.println("Pilihan : ");
            choice = input.nextInt();
            input.nextLine();

            switch (choice) {
            case 1:
                tambahLagu_3011(musikList, input);
                break;
            case 2:
                lihatPlaylist_3011(musikList);
                break;
            case 3:
                hapusLagu_3011(musikList, input);
                break;
            case 4:
                System.out.println("Keluar dari Program.");
                break;
            default:
                System.out.println("Pilihan tidak valid.");
            }
        } while (choice != 4);
        input.close();
    }
}
```

Output :

```
Console
Playlist_2511533011 [Java Application] C:\Users\Lenovo
===Playlist Musik NIM : 2511533011===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Keluar
Pilihan :
1
Masukkan Judul :
Heaven
Masukkan Penyanyi :
Bryan Adams
Masukkan Durasi(detik) :
330
Data berhasil ditambahkan!
===Playlist Musik NIM : 2511533011===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Keluar
Pilihan :
```

Program ini terdiri dari dua class yaitu Musik_2511533011 dan Playlist_2511533011 yang digunakan untuk membuat aplikasi sederhana pengelolaan playlist musik. Class Musik_2511533011 berfungsi sebagai template objek lagu yang menyimpan data judul lagu, nama penyanyi, dan durasi lagu, serta memiliki constructor untuk mengisi data dan method toString() agar data dapat ditampilkan dengan rapi. Sedangkan class Playlist_2511533011 berfungsi sebagai program utama yang menyediakan menu untuk menambah lagu ke dalam ArrayList, menampilkan seluruh daftar lagu, menghapus lagu berdasarkan judul, serta menggunakan perulangan do-while agar menu terus berjalan sampai pengguna memilih keluar. Program ini

menerapkan konsep dasar OOP seperti class, object, constructor, method, dan override.